

NoteMD — pre-pilot technical work, progress update

29 September 2026 (Supersedes the draft of 17 September, which was not sent.)

Hi Mohamed,

Nine of your ten items are now complete or code-complete. The security and safety testing you commissioned has found **seven defects**, all of them fixed. Two were serious enough that I'd want you to read those sections properly rather than skim.

There are also five decisions that need you, and I've listed them together at the end so they're in one place.

1. The testing found a critical vulnerability, before the penetration test

Any authenticated account could read any clinician's patient records. Not a theoretical weakness: a single database update, and one account could see another clinician's letters, recordings and stored audio in full.

The cause sat between two pieces of access control that each looked correct on their own. Secretary access is worked out at query time from a field on the user's own profile. The rule governing profile edits restricted *which row* a user could change, but not *which columns*. So any account could point that field at a clinician and inherit read access to their entire record set.

Two related problems came from the same gap: an account could make itself an administrator, and could write audit entries attributed to another clinician.

All three are fixed, and live in production.

Two things worth drawing out:

- **This could not have been found by reading the code.** Each policy is correct in isolation; the hole exists only in how they interact. It was found by authenticating as one user and attempting to reach another user's data against a live database — the cross-user testing in your item 5.
- **It was found before the penetration test, not during it.** The other way round, it would have been a critical finding in a report going to a Trust's information governance team, with remediation under time pressure.

The suite that found it now runs 24 hostile access attempts — manipulated record identifiers, storage access, secretary permissions against assigned and unassigned clinicians, privilege escalation, audit tampering, access after account revocation. All 24 are refused, and it writes an evidence file on every run.

2. One patient's words could reach another patient's letter

Found during your item 7. **Severity: clinical safety.**

Dictation is transcribed in ten-second segments, each sent off and appended to the transcript when it returns. Before moving to the review screen the recorder waits for outstanding segments — but only for eight seconds, then proceeds.

So a slow segment could still be in flight after the clinician had finished, generated the letter, and started recording **the next patient**. When it landed, it appended to whatever transcript was current: one patient's spoken words in another patient's transcript, and from there into their letter.

Fixed with a session token that a segment checks before its result is used. The same guard now covers the streaming path.

A second, related defect: **two consultations open in different tabs overwrote each other's crash-recovery data**. One session was lost outright, and the survivor could be offered back in the other tab under a different patient's name. Recovery data is now held per tab, and the prompt states which patient it belongs to before you restore it.

Both are the kind of defect a penetration test would not find. A tester probes access control; neither of these is an access-control failure. They came out of the functional review you scoped as the cheaper alternative to an automated browser suite — and that is where the clinical-safety defects turned out to be.

3. A letter could be emailed before any clinician had read it

Your stated boundary is that AI generates a draft, the clinician reviews and approves, and only then does the letter leave the system. Two routes crossed it:

1. The email function accepted any letter regardless of status, then marked it exported. A letter that had never left draft could be sent and recorded as exported, with no clinician action in between.
2. Where a clinician had **auto-send** switched on, the system emailed the letter *the moment the AI produced it*. Those clinicians never saw it.

Both are closed. Sending is refused for anything unreviewed, and generation no longer sends at all.

A change your users will notice: auto-send now fires when the clinician saves their review, rather than on generation. Anyone relying on it will find they must open and save each letter, at which point it sends automatically as before. Worth telling them rather than letting them discover it.

4. Three further defects, all fixed

Patient data was being written into server logs. Every server function ended by logging the error when something failed — which reads as harmless. In fact, when the database rejects a write it returns the rejected row as part of the error, so a failed letter insert wrote the patient's name, NHS number and transcript into retained logs. This was not hypothetical: the autosave problem you reported earlier this year was producing exactly this class of rejection, repeatedly, in production. Errors are now reduced to a code and a truncated message, with row data dropped entirely.

Patient identifiers were being sent to the AI provider unnecessarily. The letter prompt included the name and NHS number, and the model does nothing with them except copy them into its output. They are now replaced with placeholder tokens and reinstated on our own server. To be precise about what this is: **data minimisation, not anonymisation** — the transcript still goes to the provider and a name spoken aloud is inherent to the audio. But the structured identifiers no longer leave our systems. This is a lighter-touch version of the identifier system you declined, and deliberately avoids the wrong-patient risk you were concerned about: there is no matching step, because substitution happens on one letter using values read from that letter's own record.

Dependency vulnerabilities. 25 advisories at the start of this work, 13 of them in code that reaches the browser. Now **zero** in shipped code, with four remaining in build tooling that never reaches a user.

5. Azure OpenAI — code complete, one decision needed

The migration is built and tested against your resource. Both deployments respond correctly, and I verified transcription end to end by sending real speech and getting the sentence back verbatim.

Switching over is a **configuration change, not a release**: set two environment variables and it uses Azure; unset them and it reverts. That keeps the changeover reversible in seconds.

The decision: your instruction was to prefer a UK regional deployment. UK South does not host the models we need, so the working resource is in **France Central**, which every response header confirms. France is in the EEA and the UK recognises the EEA as adequate, so this is legally straightforward — but it is a change from the UK-only position in your specification and needs recording in the DPIA rather than assuming.

Worth asking your pilot Trust early whether they have a UK-only requirement, because that is much easier to discover now than at DTAC review.

A second point: you deployed `gpt-4o-transcribe`, which Microsoft retires on **31 December**. That is mid-pilot. `gpt-transcribe` runs until 2028. Changing transcription model means redoing clinical validation, so it is better done once, now, than twice.

I have pointed letter refinement at your existing `gpt-4o` deployment, so the missing `gpt-4o-mini` no longer blocks anything.

For now I am treating `notemd-eu` as the working resource for both environments, since it is the only one with deployments — the UK South resource is empty. That is fine for testing, but your own requirement is separate credentials per environment, so production and staging should end up with their own resources before the pilot. I will set that up once you have confirmed the residency position, since there is no point provisioning twice if the answer changes.

6. Email — working, but with a finding you should weigh

ACS email is built and verified on staging: sending, delivery status, duplicate suppression, audit metadata. Two things came out of it.

ACS Email is a retiring product

Microsoft has announced the retirement of Azure Communication Services as a standalone offering, effective **30 September 2028**, and Email is on the retired list. From **23 October 2026**, new customers cannot sign up at all — your resource predates that, so it is grandfathered. Microsoft's own guidance is to use the two-year window to migrate *off* ACS Email rather than onboard new workloads.

You asked me to proceed on ACS having been told this, and I have. It is recorded in the evidence pack so it appears as a known constraint rather than a surprise in 2028. The provider-specific code sits in one file and two call sites, so a later move is small.

Deliverability is the real issue

We set up `mail.notemd.co.uk` properly: verified domain, SPF, two DKIM keys, an explicit DMARC policy. Received headers confirm **SPF passes and DKIM passes with a 2048-bit key**.

Test letters still arrive in the junk folder of a personal Outlook mailbox. I could find no remaining configuration defect. The most likely cause is sender reputation — the domain sent its first message the same day, and large providers weight sending history heavily regardless of how well mail is authenticated. That improves over weeks and cannot be fixed with more DNS.

This matters more than it sounds. A letter filed into a recipient's junk folder is never read, while the clinician believes it was sent. NoteMD cannot see junk placement, and bounce handling is not implemented — ACS can report delivery outcomes, but wiring that up is additional work that is not in scope.

So: a letter that silently fails to reach a GP is a clinical-safety issue, not a deliverability inconvenience. If reaching a personal Outlook mailbox takes this much work, reaching NHS mail systems reliably from a newly provisioned third-party domain is a material risk.

My recommendation, in order:

1. **NHSmial or the Trust's approved secure pathway** for patient-identifiable correspondence. A Trust is likely to require this regardless.
2. **Export into the Trust's clinical record** rather than email — the workflow you described as preferred in your own item 10.

3. **Email as a convenience channel only**, with the clinician told explicitly that delivery is not confirmed.

I would not rely on emailed clinical letters from a new domain for the pilot.

7. Everything else

Audit trail. Records who did what, against which record, when, and with what outcome. Three properties are enforced by the database rather than by convention: actor and subject are recorded separately, so a secretary acting on a clinician's record is properly attributable; the log cannot be edited or deleted by anyone, including our own server credentials; and clinical content cannot enter it — an attempt to write a transcript into an audit entry is rejected outright.

Staging environment. Live, in the same region as production, with its own database, storage and credentials. Development and testing no longer run against the production clinical database.

Session and endpoint security. A configurable inactivity timeout, default 30 minutes, which **suspends itself during a recording** so a clinician is never signed out mid-consultation. Rate limiting on all seven sensitive endpoints. Security response headers including a Content Security Policy.

One result from that is worth noting for the DPIA: the browser streams consultation audio directly to the transcription provider, and the policy now names only the EU endpoint. **EU residency for transcription is therefore enforced in the browser itself** — if anything ever pointed the stream at a non-EU region, the connection would be refused rather than silently leaving the EEA. Confirmed by observation in production, not just configured.

Backups. Daily, seven days retained, stored in the same region as the database so they create no transfer outside the EEA. More importantly, a restore has been *rehearsed*: both clinical tables were deleted entirely in staging and recovered, with row counts, a content checksum and the links between letters and recordings all verified identical. A backup that has never been restored is a hypothesis.

Two points for the DPIA that are easy to leave implicit: an erased record remains inside backups for up to seven days; and once a letter has been exported, the copy in the Trust's record or a recipient's mailbox is outside NoteMD's control entirely.

Billing. While deploying, I found that the Stripe webhook had been configured outside version control, and a routine deployment silently disabled it — payment events were being refused. Now fixed, recorded in configuration, and covered by a test so it cannot recur. Stripe retries for three days, so no events should have been lost.

8. Decisions I need from you

1. **France as the AI processing location.** Confirm, and record it in the DPIA. Check whether your pilot Trust has a UK-only requirement.

2. **The transcription model.** Move to `gpt-transcribe` now, or accept changing model mid-pilot in December.
 3. **Email for clinical correspondence.** See section 6. This is the most consequential decision in this update.
 4. **The retention proposal.** Written up and waiting. One correction to its premise: retention is **not** hard-coded at ten years and has not been since August — it is already configurable, audio already defaults to thirty days, and a scheduled job enforces it. The real gap is that NoteMD has no concept of an organisation, so every setting is per user account. That is what needs building, and it is a smaller piece of work than a retention redesign.
 5. **Deleting audio once a letter is exported.** You asked for this. I would make it configurable and default it to *off*, with a short grace period. The recording is the only evidence of what was actually said; if a letter is later disputed — a wrong dose, an omitted finding — the audio distinguishes a transcription error from a clinical one. **That trade-off belongs to your Clinical Safety Officer and the hazard log, not to me.**
-

9. Production access and recovery — two things to act on

I compiled the access register from the Supabase management interface rather than asking you to gather it. It produced two findings worth acting on this week.

Neither administrative account has multi-factor authentication

Two accounts hold unrestricted access to the production clinical database: your Owner account and my Administrator account. **Neither has MFA enabled.**

NoteMD requires MFA of its clinicians. The two accounts that can read every patient record do not have it. A single password compromise on either exposes the whole dataset and bypasses every other control in place, Row Level Security included, because an organisation owner reads the database directly.

This will be raised by any competent penetration test, and it is a DSPT expectation. It costs nothing to fix: Supabase Dashboard → Account Preferences → Security, on both accounts. I will enable it on mine today; please do the same on yours.

Separately, the Owner account is a personal `yahoo.com` address rather than one on a domain the company controls. That makes account recovery dependent on a consumer mailbox outside the business, and leaves no administrative route to recover the production environment if you were unavailable. I would move Owner to an address on `notemd.co.uk`, which already runs on Microsoft 365.

Point-in-time recovery is not enabled

I checked rather than asked: production takes daily backups, holds eight of them, archives write-ahead logs, and sits in `eu-west-1` — but point-in-time recovery is switched off.

That means the recovery point objective is up to 24 hours. A failure late in the working day would lose that day's consultations, transcripts and letters. Those cannot be reconstructed: the consultation is over and the audio is not retained indefinitely.

Point-in-time recovery is a paid Supabase add-on that reduces the objective to roughly two minutes. It is your decision, but the cost is modest against losing a day of clinical documentation, and a pilot organisation is likely to ask what our recovery point objective is.

Already resolved

Deepgram now runs under your own account rather than mine. The retired Google Cloud Run transcription service sits in my personal cloud account, holds no current clinical data, and I am decommissioning it.

10. Where this leaves the programme

Nine of ten items complete or code-complete. Item 10 — final documentation and the evidence pack — is deliberately last, because it has to describe the final architecture once Azure and email are settled.

The automated suite now stands at **300 tests**, plus 24 hostile-access tests that run against a live database. Each defect above has a test that reproduces the original failure, so none of them can return silently.

Seven defects found and fixed: one critical cross-tenant data exposure, two privilege escalations, two clinical-safety defects, one log disclosure, one data-loss and wrong-patient recovery bug.

Nothing here changes the sequence you set out: development, internal review, automated testing, final documentation, freeze, then the independent penetration test.

Happy to talk any of this through.

Chris